Regression

Dr. Vimal K. Shrivastava
Assistant Professor (II)
School of Electronics Engineering
KIIT Deemed to be University,
Bhubaneswar

Introduction

- Regression is a technique used to predict dependent variable given a set of independent variables.
- Regression analysis is a form of predictive modelling technique which investigates the relationship between a dependent and independent variable(s).
- It is used to predict the numeric data instead of labels.
- Output is continuous value.

Examples

- Estimating income by age.
- Estimating price of a house by its size.
- Estimating price of a car by its model and purchase year.
- Estimating share prices for market analysis.

Linear Regression

- Linear regression can be thought of finding relationship between two things i.e. Dependent variable (y) and independent variable (X) using a straight line.

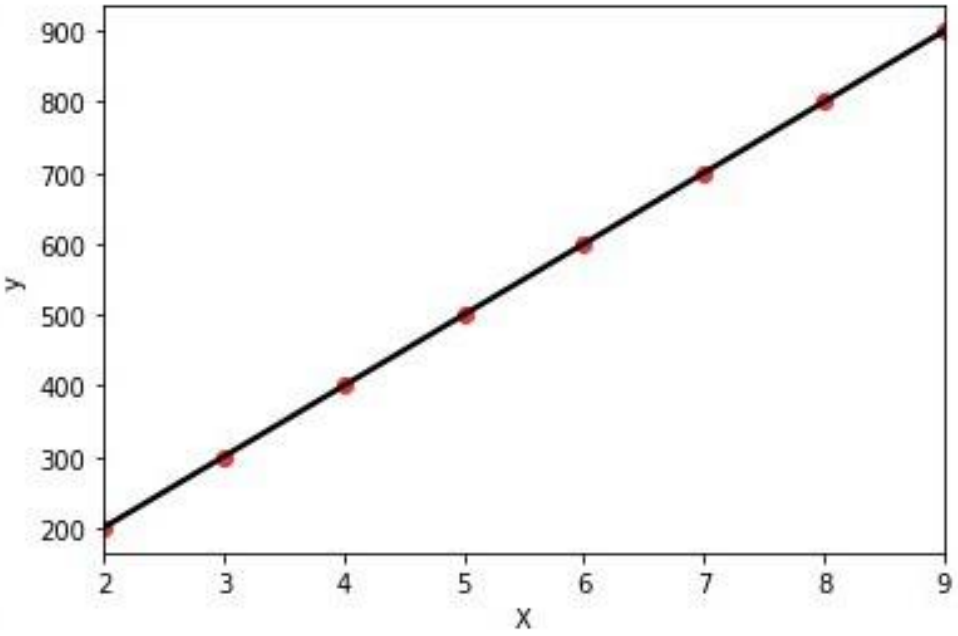
- **Simple Linear Regression** — In simple linear regression, there is only one independent variable (X) and based on that dependent variable (y) is predicted.

- **Multiple Linear Regression** — In multiple linear regression, there are multiple independent variables (X1, X2…,Xn) and based on that dependent variable (y) is predicted.

Simple Linear Regression using Gradient Descent

- In below data there is one independent variable (X) and y is dependent variable. The task is to identify y for new value of X.

X	y
2	250
3	350
4	450
5	550
6	650
7	750
8	850
9	950



Ploting this points for X and y

- In above plot red dots are the data points and as we need to find linear relationship, hence we have connected them with a line.

- It is well known that equation for a line is $\mathbf{y = mX + c}$, which can also be written as
  - $\mathbf{Y=\beta 0+\beta 1X1}$

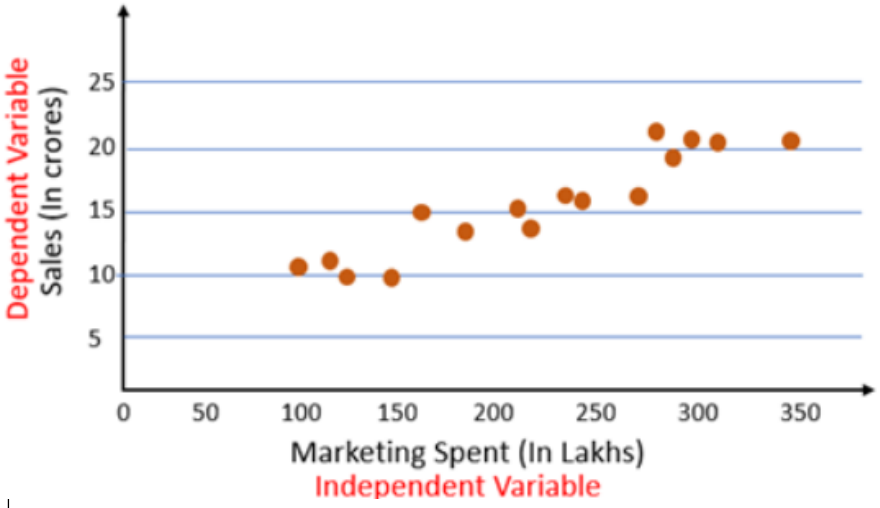
  where,
  - $\beta 0$ is C from original equation i.e. intercept.
  - $\beta 1$ is m from original equation i.e. slope.

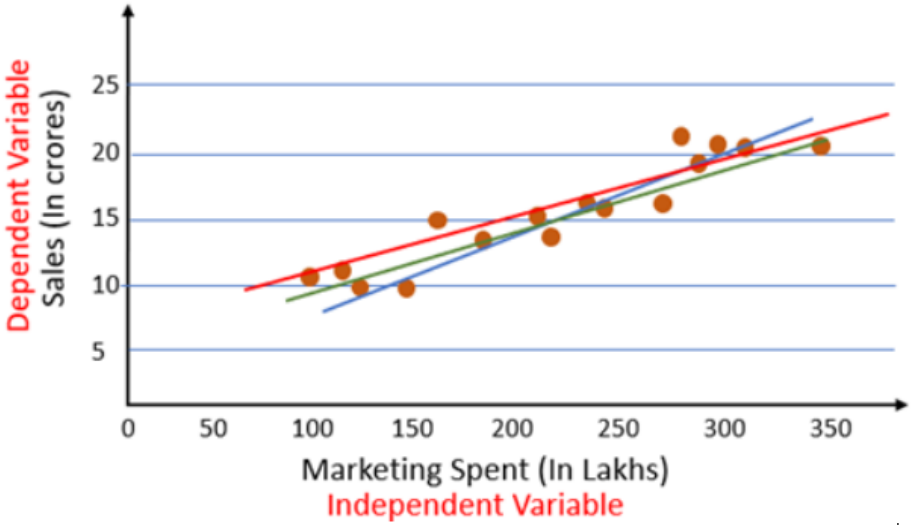
- So, if this equation of line is completed, Y can be predicted for future X.

- Here, from past few data, X and y are known. The aim is to find values of $\beta 0$ and $\beta 1$ to complete our equation.

- For above data, linear equation would be Y=50+100X. This is what the linear regression algorithm would try to get. Now for any value of X we can predict Y.

- But, would the real world data be this simple ? Obviously not. It would be like below or even more complex.



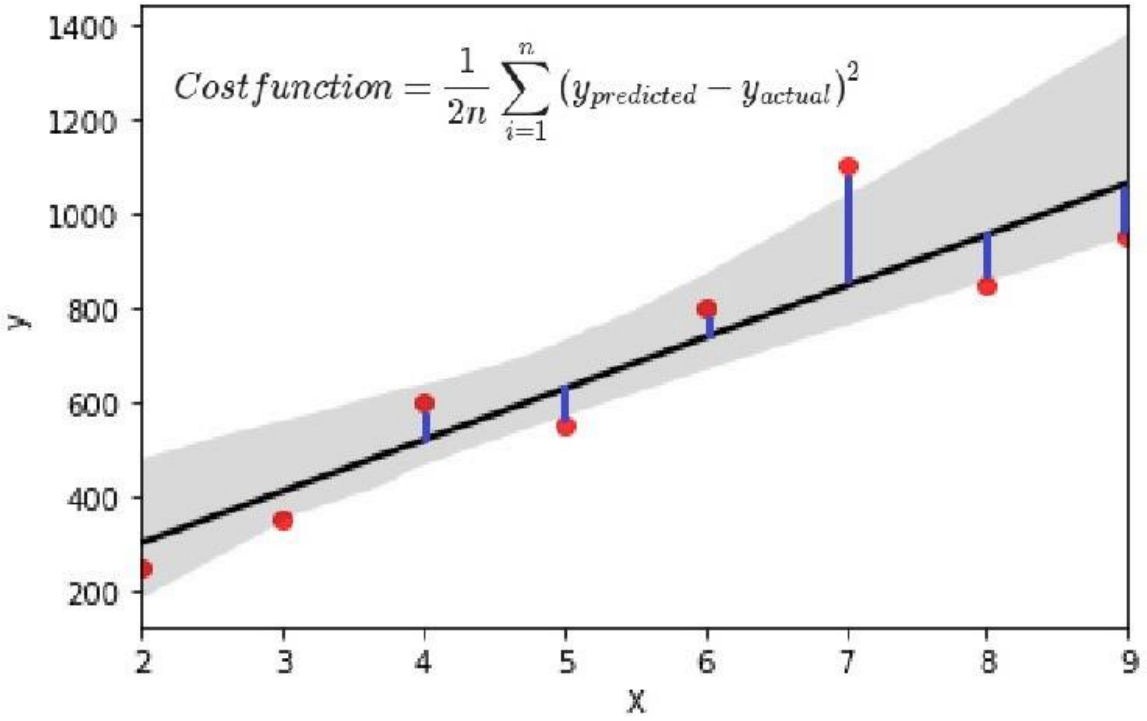


- In above plots, a simple line passing through all the points can not be possible. Hence, objective would be to find the "**best fit line**".

- There can be various lines possible, but Linear regression model try to make a line such that the vertical distance between the line and the data points (residuals) is as small as possible. This is called "fitting the line to the data" and will be our **'best fit line'**.

- **Cost Function:**
- The actual points and the prediction given by best fit line will have variation and this is cost occurred (loss) at that point.

- Cost occurred is the difference between actual point and predicted point (i.e. vertical distance). The total cost function for all the points is called **sum of squared distance** and is given by:



- Now, we need to find best value for $\beta 0$ and $\beta 1$ for $Y=\beta 0+\beta 1 X1$.
- Here, $\beta 0$ is intercept, assuming that best fit line passes through origin it's value will be 0 and the equation would be $y=\beta 1 X1$. (however $\beta 0$ can have any value but to understand easily we have taken 0).
- Now, $\beta 1$ value should be such that we get minimum cost.
- Checking cost function values for different values of $\beta 1$(slope) and X for below 4 points: (1,1), (2,2), (3,3), (4,4).
- For each point, we have X and y(actual). Using equation $y=\beta 1 X$, y(predicted) will be calculated for different values of $\beta 1$ and then get the cost occurred for each value of $\beta 1$.

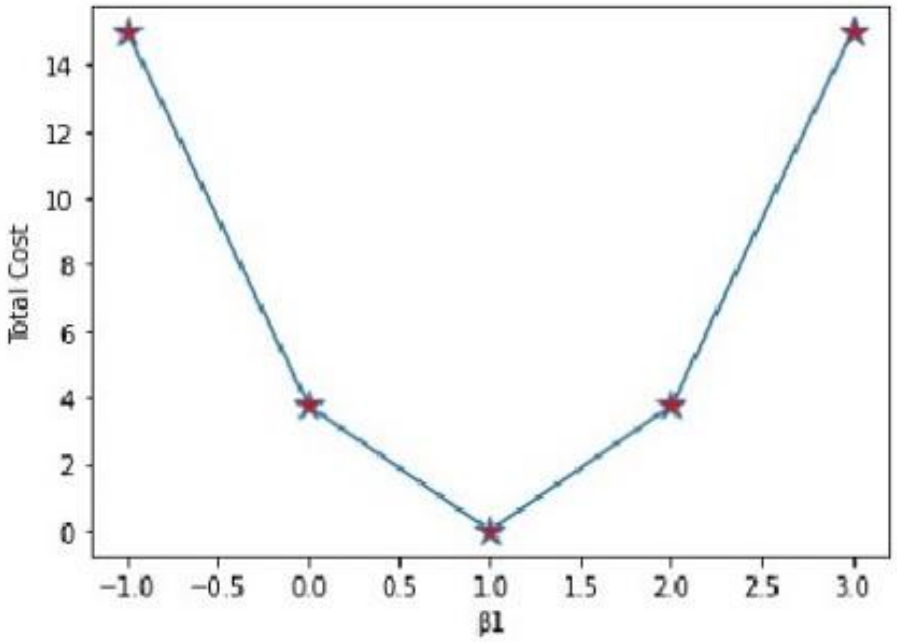
For β1 = -1			
X	Y_actual	Y_predicted	Total Cost
1	1	-1	15
2	2	-2	
3	3	-3	
4	4	-4	

For β1 = 0			
X	Y_actual	Y_predicted	Total Cost
1	1	0	3.75
2	2	0	
3	3	0	
4	4	0	

For β1 = 1			
X	Y_actual	Y_predicted	Total Cost
1	1	1	0
2	2	2	
3	3	3	
4	4	4	

For β1 = 2			
X	Y_actual	Y_predicted	Total Cost
1	1	2	3.75
2	2	4	
3	3	6	
4	4	8	

For β1 = 3			
X	Y_actual	Y_predicted	Total Cost
1	1	3	15
2	2	6	
3	3	9	
4	4	12	



- From above, it is clear that $\beta 1 = 1$ is the best value as predicted points are same as actual.

- In short, for lowest value of cost function, optimal value of $\beta 1$ can be obtained.

- To get the lowest value of cost function **Gradient Decent** algorithm is used to get global minima.

$$Cost\ Function(MSE) = \frac{1}{n}\sum_{i=0}^{n}(y_i - y_{i\,pred})^2$$

Replace $y_{i\,pred}$ with $mx_i + c$

$$Cost\ Function(MSE) = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

- Our goal is to minimize the cost as much as possible in order to find the best fit line.

- We are not going to try all the permutation and combination of m and c (inefficient way) to find the best-fit line. For that, we will use **Gradient Descent Algorithm**.

- Use β_0 and β_1 in place of C and m.

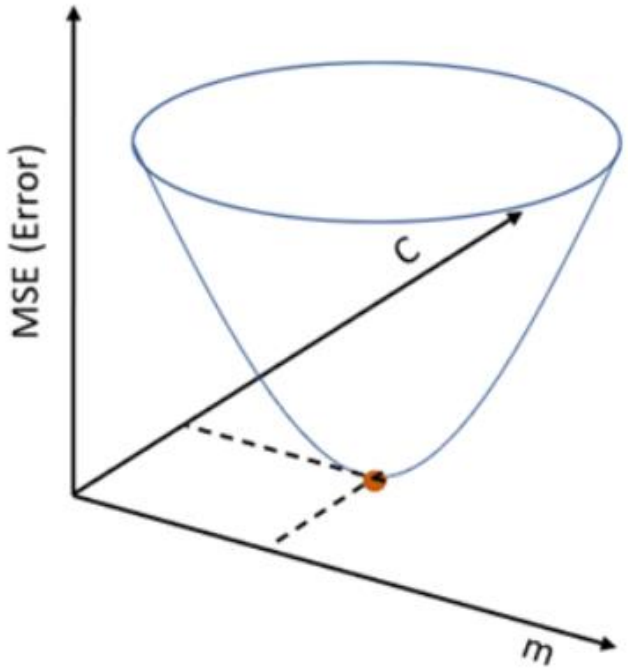
Gradient Descent

- Gradient descent is an optimization algorithm that is used for training a model.
- It is used to find the values of a function's parameters that minimize a cost function as far as possible.
- It is used for finding minimum of a differentiable function.
- **WHAT IS A GRADIENT?**
- A gradient measures how much the output of a function changes if you change the input a little bit.
- In mathematical terms, a gradient is a partial derivative with respect to its inputs.

Gradient Descent Algorithm

- Gradient Descent is an algorithm that finds the best-fit line for a given training dataset in a smaller number of iterations.

- If we plot m and c against MSE (cost), it will acquire a bowl shape (As shown in the diagram below).



Step by Step Algorithm

- **Step 1:** Let initialize m and c with small random value.
- Let L be our learning rate. Learning rate gives the rate of speed where the gradient moves during gradient descent.
- It varies in a range from 0 to 1. It could be a small value like 0.01.
- **Step 2:** Calculate the partial derivative of the Cost function with respect to m i.e. with little change in m how much Cost function changes. Let's denote it as Dm.

$$D_m = \frac{\partial(Cost\ Function)}{\partial m} = \frac{\partial}{\partial m}\left(\frac{1}{n}\sum_{i=0}^{n}(y_i - y_{i\,pred})^2\right)$$

$$= \frac{1}{n}\frac{\partial}{\partial m}\left(\sum_{i=0}^{n}(y_i - (mx_i + c))^2\right)$$

$$= \frac{1}{n}\frac{\partial}{\partial m}\left(\sum_{i=0}^{n}(y_i^2 + m^2x_i^2 + c^2 + 2mx_ic - 2y_imx_i - 2y_ic)\right)$$

$$= \frac{-2}{n}\sum_{i=0}^{n}x_i(y_i - (mx_i + c))$$

$$= \frac{-2}{n}\sum_{i=0}^{n}x_i(y_i - y_{i\,pred})$$

- Use β_0 and β_1 in place of C and m.

- Similarly, let's find the partial derivative with respect to c i.e. with little change in c how much Cost function changes. Let's call it as Dc.

$$D_c = \frac{\partial(Cost\ Function)}{\partial c} = \frac{\partial}{\partial c}\left(\frac{1}{n}\sum_{i=0}^{n}(y_i - y_{i\,pred})^2\right)$$

$$= \frac{1}{n}\frac{\partial}{\partial c}\left(\sum_{i=0}^{n}(y_i - (mx_i + c))^2\right)$$

$$= \frac{1}{n}\frac{\partial}{\partial c}\left(\sum_{i=0}^{n}(y_i^2 + m^2x_i^2 + c^2 + 2mx_ic - 2y_imx_i - 2y_ic)\right)$$

$$= \frac{-2}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))$$

$$\frac{-2}{n}\sum_{i=0}^{n}(y_i - y_{i\,pred})$$

- Use β_0 and β_1 in place of C and m.

- **Step 3**: Now, update the current values of m and c using the following equation:

$$m = m - LD_m$$

$$c = c - LDc$$

- where, L is the learning rate.

- In this way, we move towards a negative gradient i.e. move towards **minimum** of cost function.

- **Step 4:** We will repeat this process until our Cost function is very small (ideally 0).

- Use β_0 and β_1 in place of C and m.

Summary

- **Step 1:** Initialize β_0 and β_1 with small random value.
- **Step 2:** Initialize learning rate (L).
- **Step 3:** Compute

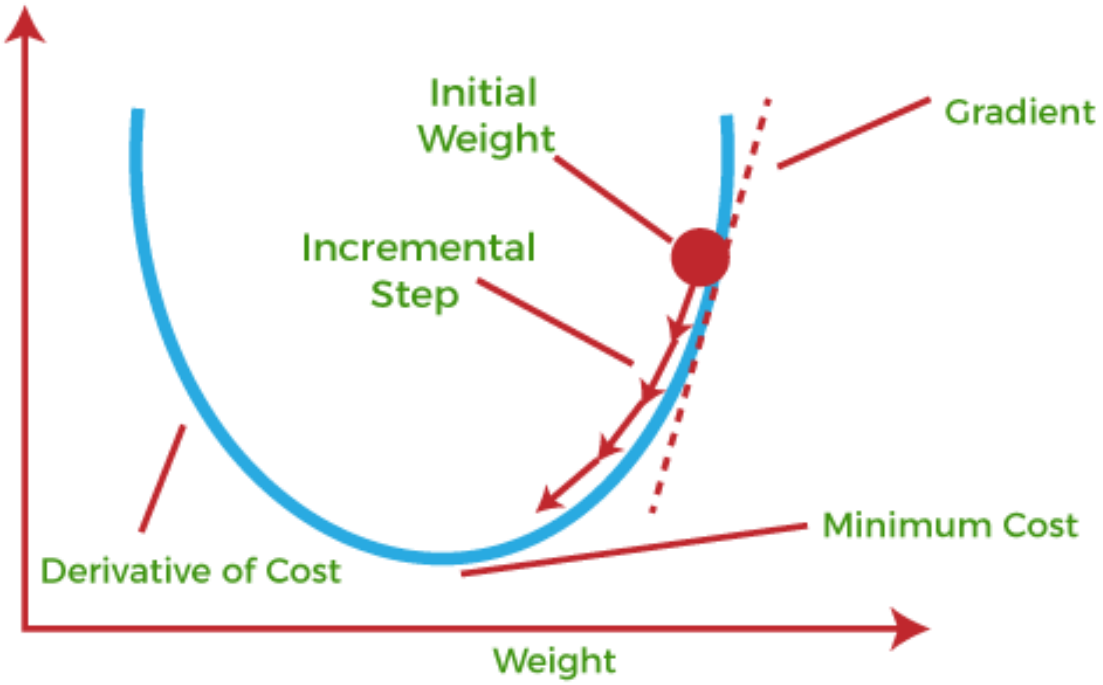
$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{n}\sum_{i=1}^{n} x_i(y_i - \hat{y}_i)$$

- **Step 4**: Update the values of β_0 and β_1

$$\beta_0 = \beta_0 - \mathrm{L}\frac{\partial J}{\partial \beta_0} \qquad \beta_1 = \beta_1 - \mathrm{L}\frac{\partial J}{\partial \beta_1}$$

- **Step 5:** Repeat step 3 and step 4 until the Cost function becomes very small.



- So, Gradient Descent Algorithm gives optimum values of m and c of the linear regression equation. With these values of m and c, we will get the equation of the best-fit line and ready to make predictions.

https://www.analyticsvidhya.com/blog/2021/04/gradient-descent-in-linear-regression/

Importance of Learning Rate

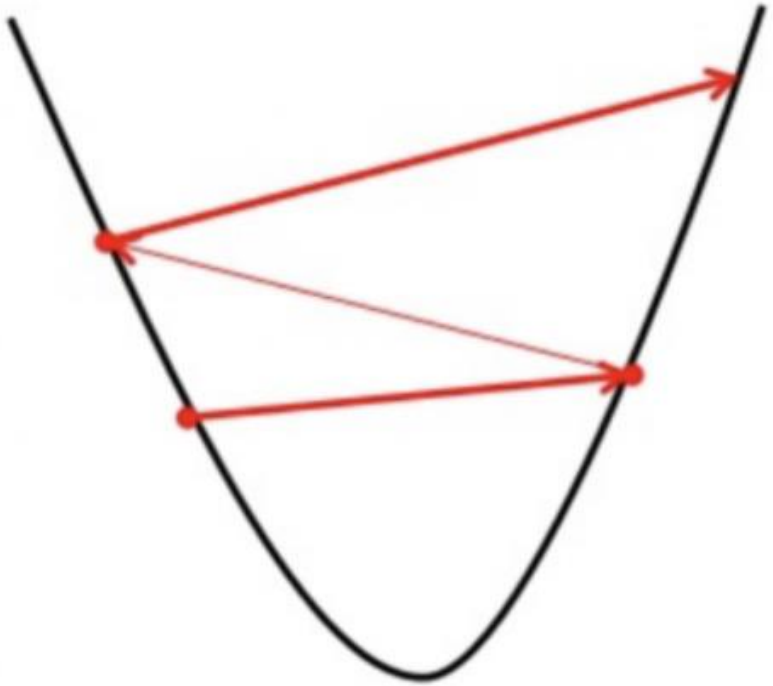
- Learning rate decides how fast or slow we will move towards the optimal parameter.

- If we set the learning rate to too high, the steps it takes will be too big, it may not reach the minimum point because it bounces back and forth between the function of gradient descent.

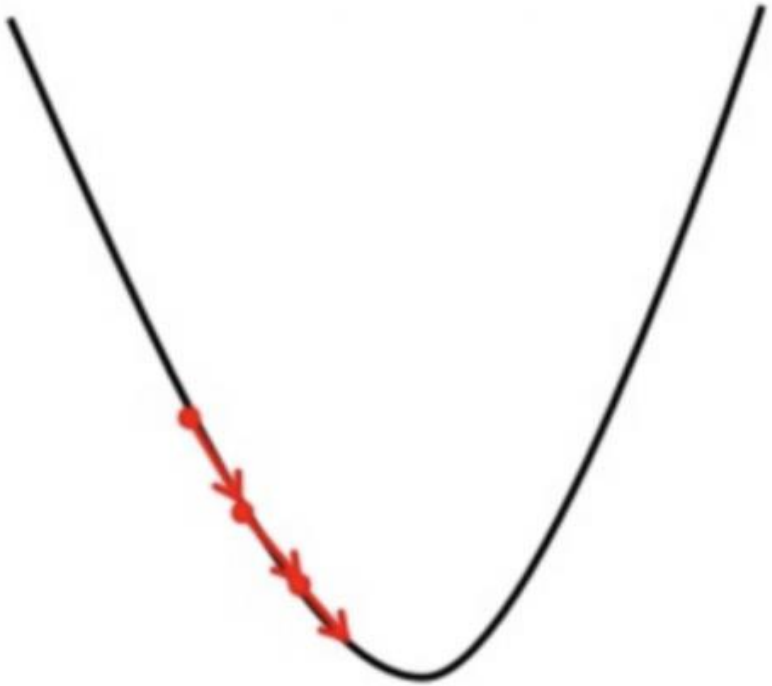
- If we set the learning rate to a very small value, gradient descent will eventually reach the minimum point but that may take a long time (computationally complex).

- Therefore, we must set the learning rate to an appropriate value, which is neither too low nor too high.

Big learning rate



Small learning rate



Multiple Linear Regression using Gradient Descent

- In the case of multiple linear regression, there could be *n* number of independent variables (input features).

- Each training example will have *n* number of columns:

$$\vec{X}^{(i)} = [x_1, x_2, x_3, \ldots, x_n]$$

- So, the linear model in the case multiple linear regression is the expanded version of the simple regression model.

$$\hat{Y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

- The coefficient values are generally the weights of that feature. It shows importance of that feature for predicting dependent variable (Y).

- So, we can write the above model in a vector form as shown below.

$$\hat{Y} = f_{\beta_0,\vec{\beta}}\left(\vec{X}^i\right) = \beta_0 + \vec{\beta}\vec{X}^i$$

- Then, cost function of this model can be written as below.

$$J(\beta_0, \vec{\beta}) = \frac{1}{2n}\sum_{i=1}^{n}\left(f_{\beta_0,\vec{\beta}}\left(\vec{X}^i\right) - y_i\right)^2$$

- Partial derivative of cost function with respect to β_0 and jth coefficient β_j.

$$\frac{\partial}{\partial\beta_j} J(\beta_0, \vec{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i - (\beta_0 + \vec{\beta}\vec{X}^i)\right)x_j^i$$

$$\frac{\partial}{\partial\beta_0} J(\beta_0, \vec{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i - (\beta_0 + \vec{\beta}\vec{X}^i)\right)$$

- Now, we can iteratively update the each feature's weight until the convergence.

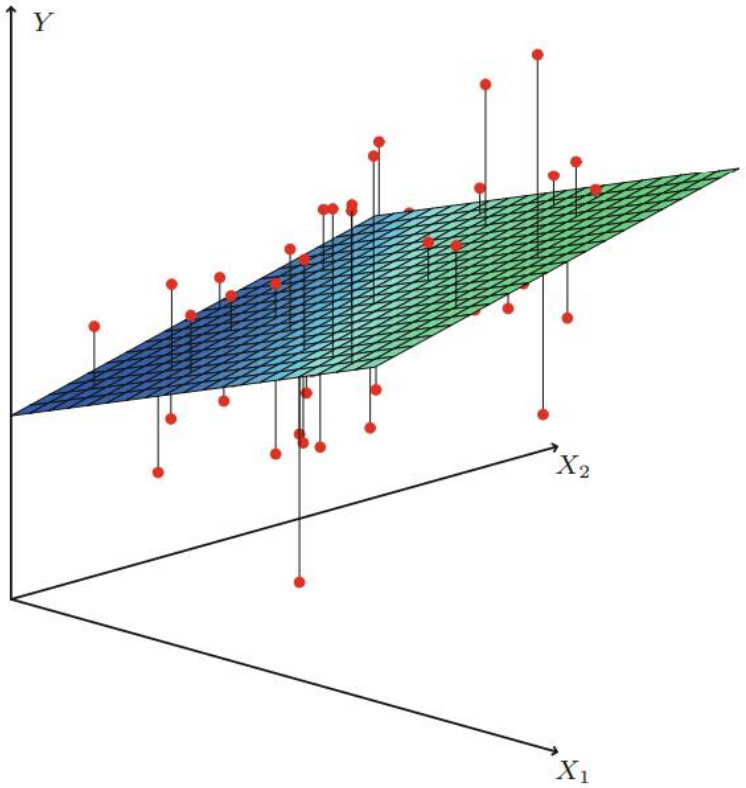
$$\beta_j = \beta_j - \mathrm{L}\frac{\partial}{\partial \beta_j} J(\beta_0, \vec{\beta})$$
$$\beta_0 = \beta_0 - \mathrm{L}\frac{\partial}{\partial \beta_0} J(\beta_0, \vec{\beta})$$

- So we can rewrite the multiple linear regression algorithm with expanded version of partial derivatives

$$\beta_j = \beta_j - \mathrm{L}\frac{1}{n}\sum_{i=1}^{n}\left((\beta_0 + \vec{\beta}\vec{X}^i) - y_i\right)x_j^i$$

$$\beta_0 = \beta_0 - \mathrm{L}\frac{1}{n}\sum_{i=1}^{n}\left((\beta_0 + \vec{\beta}\vec{X}^i) - y_i\right)$$

- In a three-dimensional setting, where there are two independent variables and one dependent variable, the regression line transforms into a plane.



- For more than two independent variables, the estimated regression equation yields a hyperplane.

Types of Gradient Descent

- There are three popular types of gradient descent technique that mainly differ in the amount of data they use.

- **Stochastic Gradient Descent (SGD):**
- It calculates the error for each training data and adjusts the parameters immediately.
- If we have 100 training data points, the SGD adjusts the parameters 100 times.

- **Batch Gradient Descent:**
- It calculates the error for each data point within the training dataset, but update the parameters only after all training examples have been evaluated.
- This method uses all of the training data and updates only once.

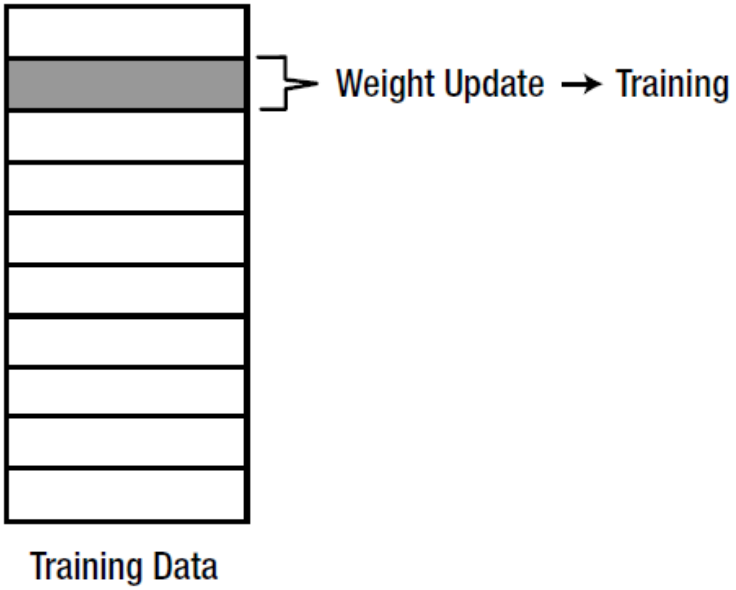
- **Mini-Batch Gradient Descent:**

- The mini batch method is a blend of the SGD and batch methods.

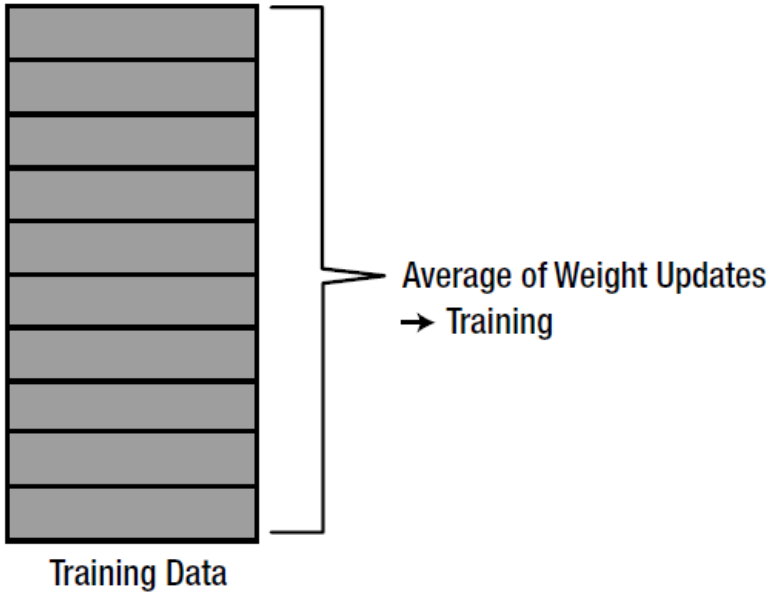
- It simply splits the training dataset into small batches and performs an update for each of those batches.

- Therefore, it calculates the parameter updates of the selected data and trains the model with the averaged parameter update.

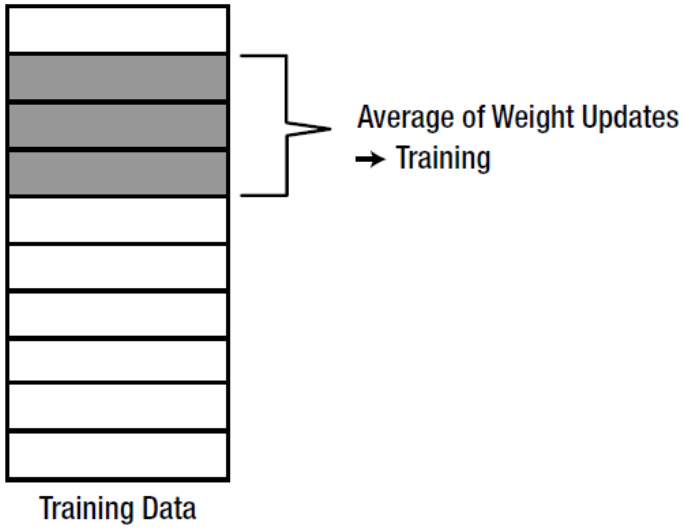
- For example, if mini-batch size is 20 and total data points are 100. In this case, a total of five weight adjustments are performed to complete the training process for all the data points (100/20=5).



Stochastic Gradient Descent



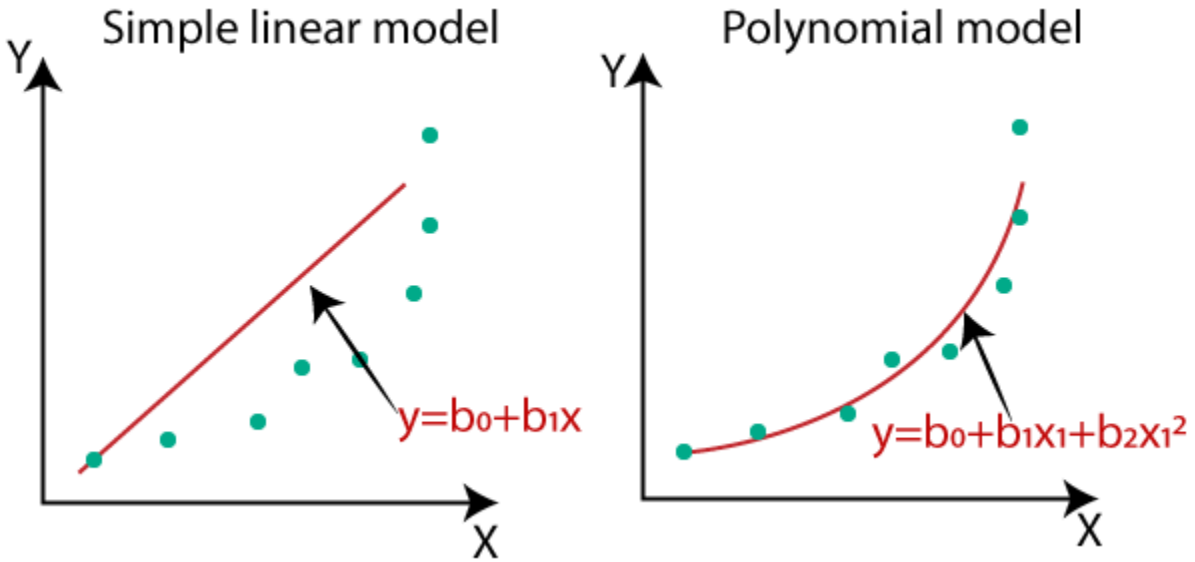
Batch Gradient Descent



Mini-Batch Gradient Descent

- **Assumptions in Linear Regression:**
- It is clear is that linear regression is a simple approach to predict based on a data that follows a linear trend. The algorithm fails if there is a curved data set.

- **Polynomial Regression:** Due to the non-linear relationship between dependent and independent variables, we add some polynomial terms to linear regression to convert it into Polynomial regression.

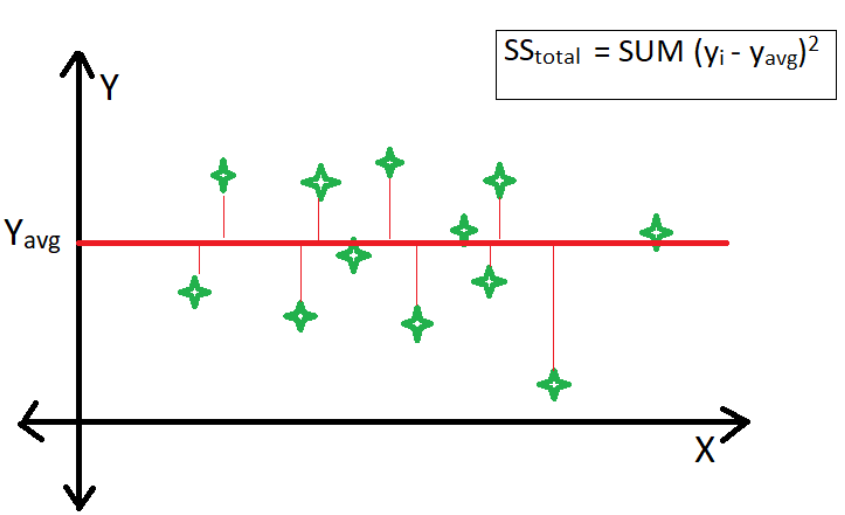


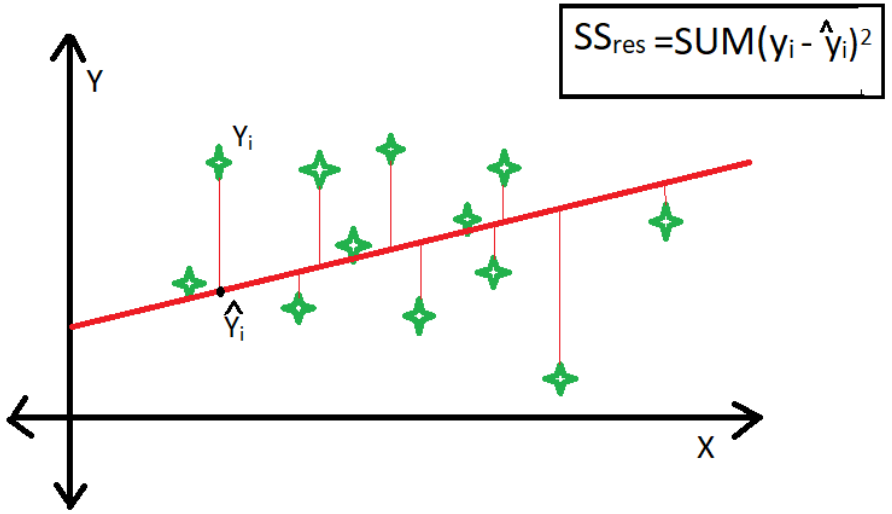
Evaluation of Regression Model

- **R2 Score:**
- R2 Score is also called Coefficient of determination.
- It is used to evaluate the performance of a linear regression model.
- It is a statistical measure that represents the goodness of fit of a regression model.
- The ideal value for R2 score is 1. The closer the value to 1, the better is the model fitted.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- R2-Score (R-square) is a comparison of residual sum of squares (SSres) with total sum of squares (SStot).



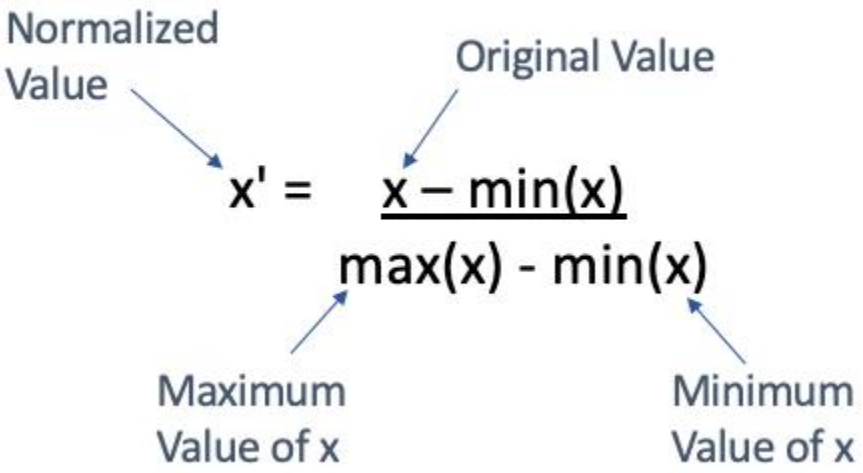


https://www.geeksforgeeks.org/ml-r-squared-in-regression-analysis/

Normalization

- Normalization is the process of converting an actual range of values of numerical features into a standard range of values, typically in the interval of [-1, 1] or [0, 1].

- It is also known as min-max normalization.



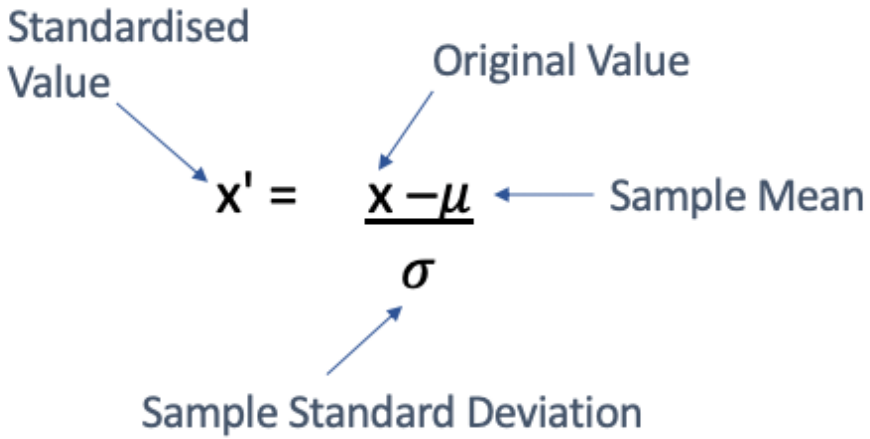
- **Why do we Normalize?**
- This is important for algorithms that are sensitive to the scale of input features, such as gradient-based optimization algorithms used in many machine learning models (e.g., linear regression, neural networks). If features are on different scales, it might lead the algorithm to give more importance to features with larger scales.

- Gradient-based optimization algorithms, like gradient descent, converge faster when dealing with normalized data.

- Normalization can lead to improved generalization and performance of the model.

Standardization

- Standardization is the procedure during which the feature values are rescaled so that they have the properties of a standard normal distribution with mean = 0 and standard deviation from the mean = 1.
- The formula for standardization (z-score normalization) of a feature X is given by:



Normalization

1. Outputs values within a specific range, often [0, 1] but can be adjusted to any desired range.
2. Sensitive to outliers, as extreme values can disproportionately affect the range.
3. Preserves the original scale of the data, making it more interpretable in terms of the original units.
4. Assumes that the data is relatively uniformly distributed across the specified range.

Standardization

1. Produces values with a mean of 0 and a standard deviation of 1. The range is not constrained, and values can be positive or negative.
2. Less sensitive to outliers due to the use of the mean and standard deviation.
3. Transforms data into a standardized form. The resulting values are in terms of standard deviations from the mean.
4. Assumes that the data follows a normal distribution or is approximately normally distributed.

Bias-Variance Tradeoff

- **What is bias?**
- Bias is the difference between the predictions on training data of model and the correct value.
- Model with high bias pays very little attention to the training data and oversimplifies the model.
- It always leads to high error on training data and test data.

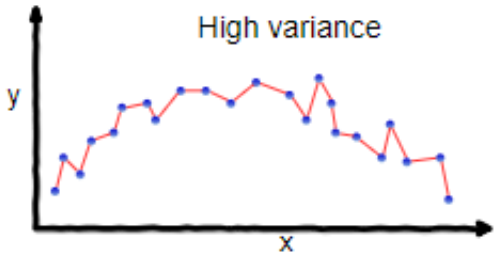
- **What is variance?**
- Variance is the variability of model prediction in training and test data.
- Model with high variance pays a lot of attention to training data and does not generalize on the test data.
- As a result, such models perform very well on training data but has high error rates on test data.

Underfitting

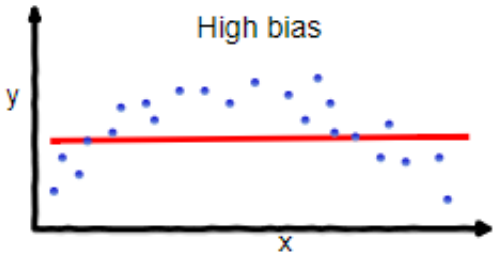
- It happens when a model unable to capture the underlying pattern of the training data.
- When the model has a high error rate in the training data, we can say the model is underfitting.
- These models usually have high bias.
- It happens when we have very less amount of data to build an accurate model or when we try to build a linear model with a nonlinear data.

Overfitting

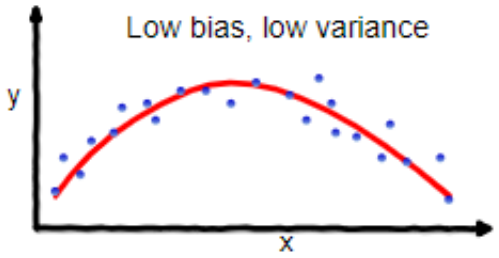
- When the model has a low error rate in training data but a high error rate in testing data, we can say the model is overfitting.
- This usually occurs when the number of training samples is too high or the hyper-parameters have been tuned to produce a low error rate on the training data.
- These models have low bias and high variance.
- These models are very complex which are prone to overfitting.



overfitting

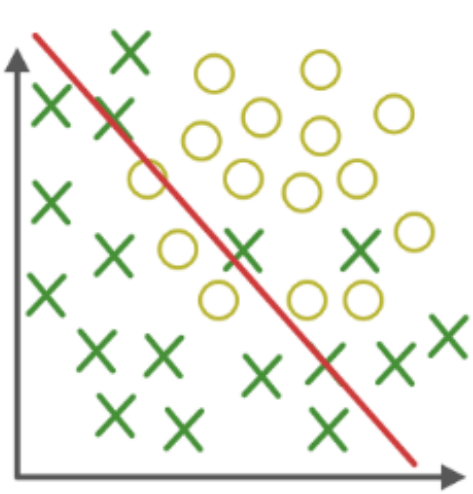


underfitting

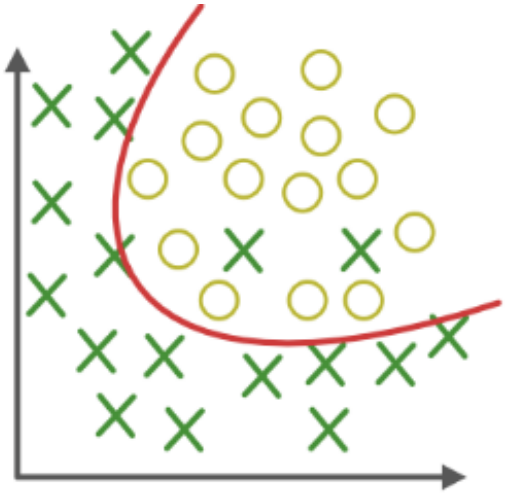


Good balance

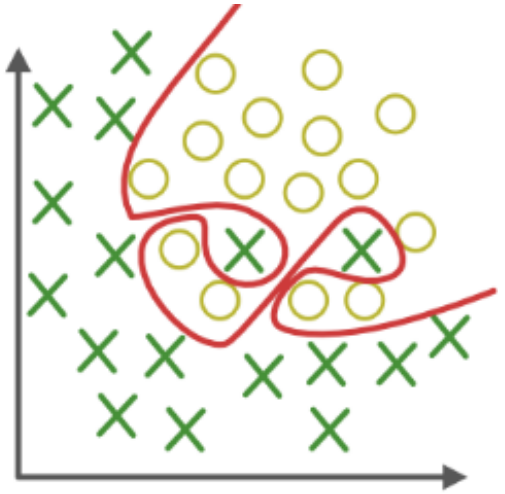
Regression Example



Under-fitting



Appropirate-fitting



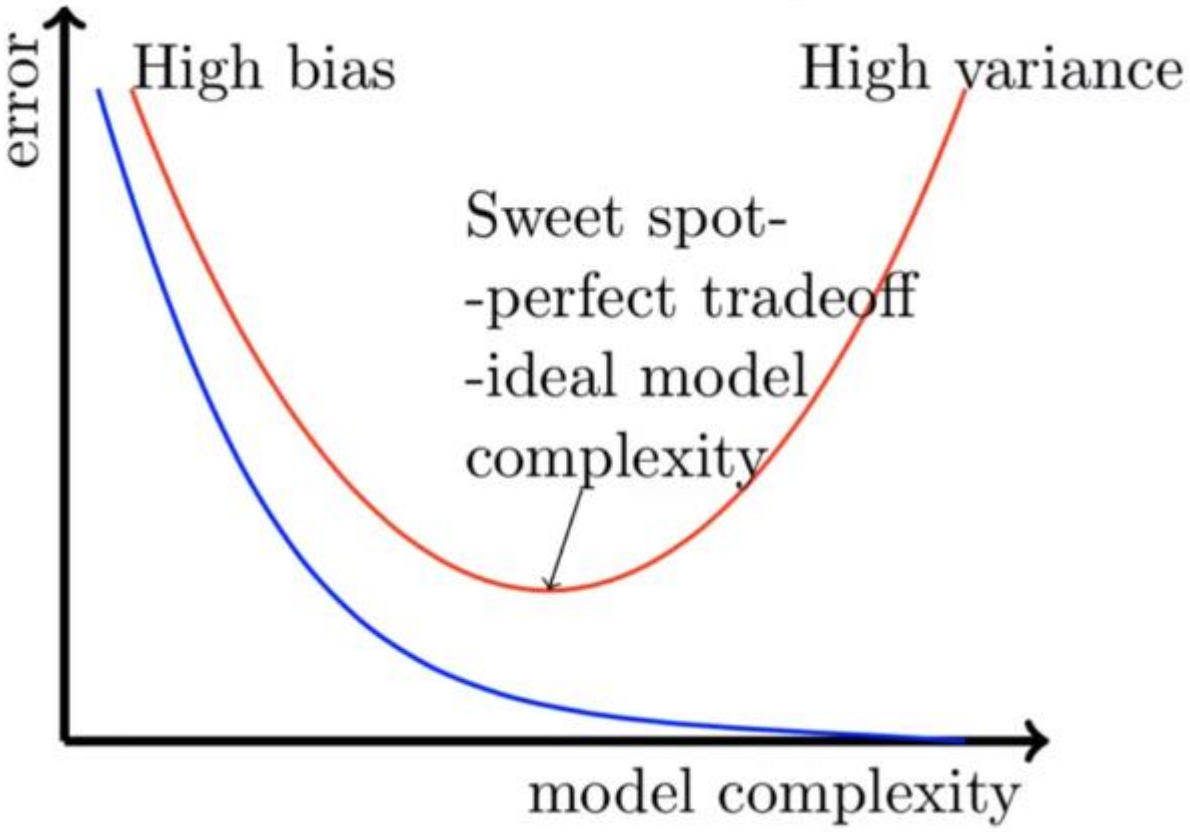
Over-fitting

Classification Example

Why is Bias Variance Tradeoff?

- If our model is too simple and has very few parameters then it may have high bias and low variance.
- On the other hand if our model has large number of parameters then it's going to have low bias and high variance.
- So we need to find the right balance without overfitting and underfitting the data.
- This tradeoff in complexity is why there is a tradeoff between bias and variance.
- An algorithm can't be more complex and less complex at the same time.

Bias-Variance Tradeoff



Blue Curve – Training

Red Curve - Testing

- When the model's complexity is too low, *i.e.* a simple model, the model won't be able to perform well on the training data nor on the testing data, therefore it's underfit.

- As the complexity of the model increases, the model performs well on the training data but it doesn't perform well on the test data and therefore it's overfit.

- At the sweet spot, the model has a low error rate on the training data as well as the test data, therefore, that's the ideal model.

- Let's take 3 examples to understand Overfitting, Underfitting and Balanced Model.

- A model with training error : 30% and test error : 30%
  High Training Error — High Bias
  Less difference between training and test error: Low Variance
  This is an Underfitted model.

- A model with training error : 2% and test error : 20%
  Less Training Error — Low Bias
  High difference between training and test Error — High Variance
  This is an Overfitted Model.

- A model with training error : 4% and test error : 3%
  Less Training Error — Low Bias
  Less difference between training and test error— Low Variance
  This is a Balanced model.

Regularization

- Regularization is a technique used in machine learning to prevent overfitting and improve the generalization performance of a model.

- It avoids overfitting by adding a penalty to the model's loss function:

$$\text{Regularization} = \text{Loss Function} + \text{Penalty}$$

- There are two commonly used regularization techniques:
  - L2 regularization
  - L1 regularization

L2 Regularization (Ridge Regression)

- A regression model that uses the L2 regularization technique is called ridge regression.

- Ridge regression adds the "squared magnitude" of the coefficient as a penalty term to the loss function(L).

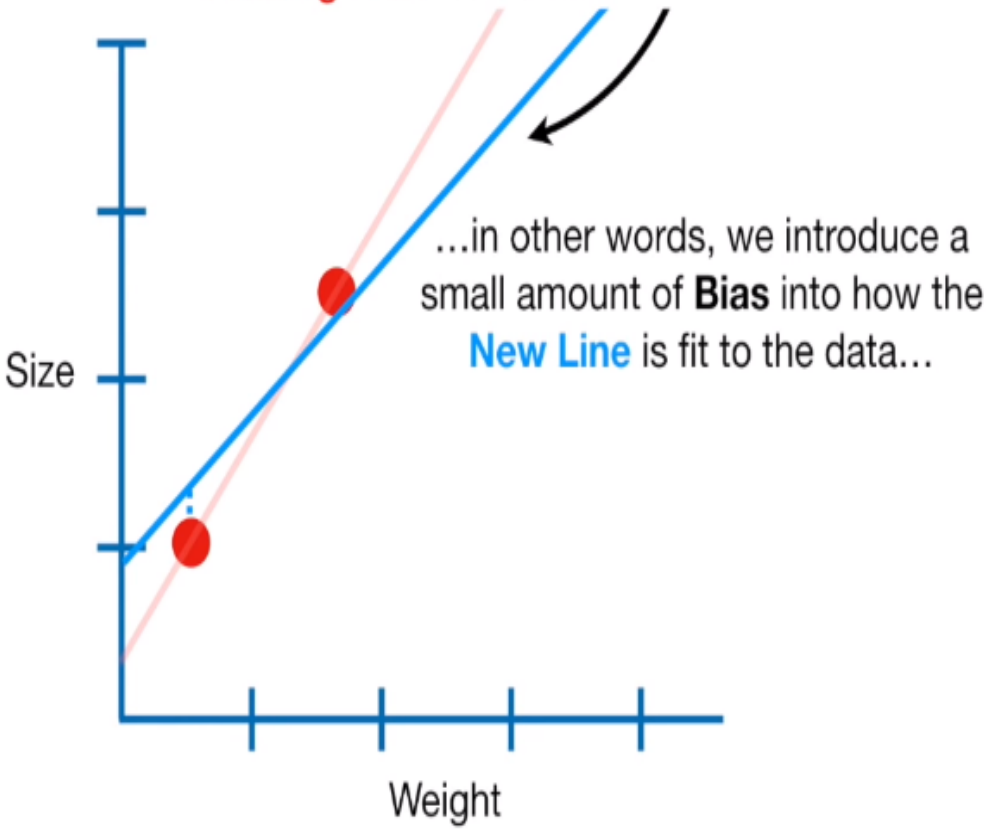
$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{i=1}^{m} w_i^2$$

- where,
- λ controls the strength of regularization. It is the hyper-parameter whose value is optimized for better results.
- w_i are the model's weights (coefficients).

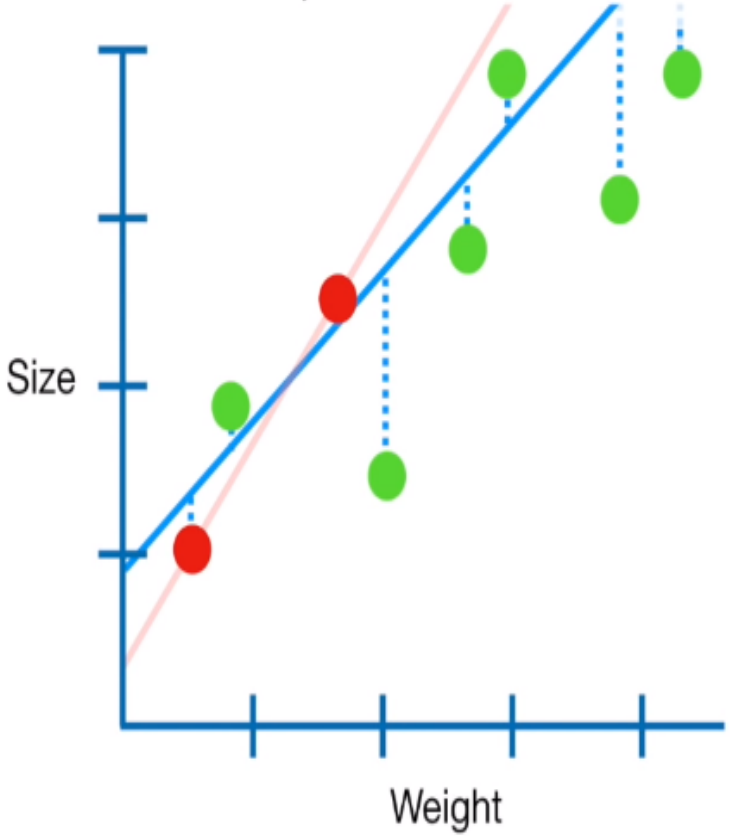
- By increasing λ, the model becomes underfit.
- By decreasing λ, the model becomes overfit.
- with $\lambda = 0$, the regularization term will be eliminated.

The main idea behind **Ridge Regression** is to find a **New Line** that doesn't fit the **Training Data** as well...

...in other words, we introduce a small amount of **Bias** into how the **New Line** is fit to the data...



In other words, by starting with a slightly worse fit, **Ridge Regression** can provide better long term predictions.



L1 Regularization (Lasso Regression)

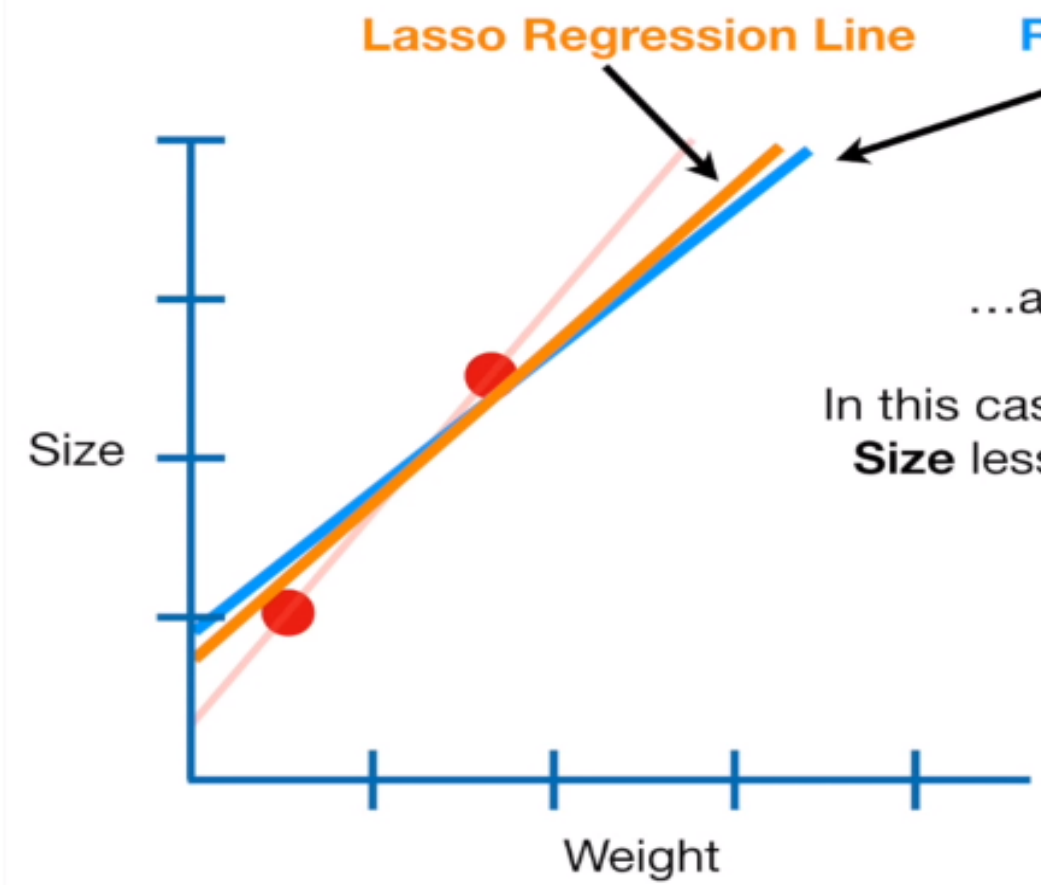
- A regression model which uses the L1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression.

- Lasso Regression adds the "absolute value of magnitude" of the coefficient as a penalty term to the loss function(L).

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} |w_i|$$

- where,
- λ controls the strength of regularization. It is the hyper-parameter whose value is optimized for better results.
- w_i are the model's weights (coefficients).

- Lasso regression also helps us achieve feature selection by penalizing the weights to approximately equal to zero if that feature does not serve any purpose in the model.



…and they do similar things.

In this case, they make our predictions of **Size** less sensitive to this tiny **Training Dataset**.